A LECTURE ON

RL Environments

for Large Language Models

"Environments are synthetic data engines, RL trainers, and eval harnesses — the same artifact, three jobs."

Where we are.

DONE SO FAR

CLASS 9a	CPT on SEC filings	✓
CLASS 9b	SFT on financial Q&A	✓
CLASS 10a	RLHF / DPO	✓
CLASS 10b	RLVR & reasoning models	✓

TODAY

The thing all of those had in common.

The claim.

An environment is the same artifact used for —

01 **Evaluation**

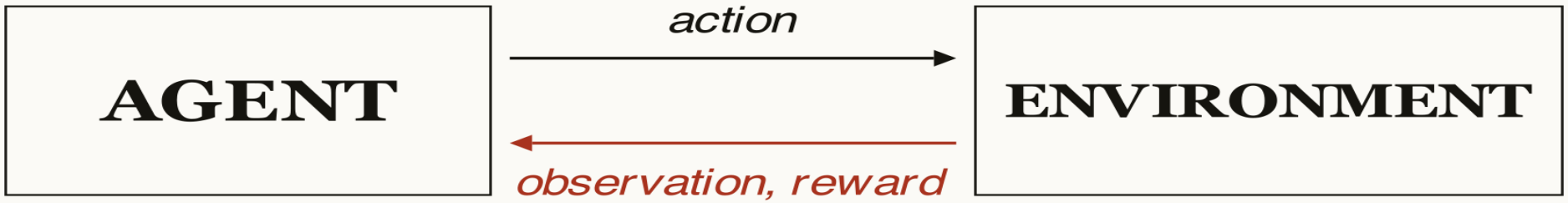
02 **RL training**

03 **Synthetic data generation**

04 **Agent harness experimentation**

Build it once. Use it four ways.

A quick RL refresher.



Goal — learn a policy that maximizes cumulative reward.

Sutton & Barto, 1998

Everything in modern LLM RL training is a special case of this loop.

From classical RL to LLM-RL.

CLASSICAL RL	LLM-RL
State	*The conversation so far*
Action	*Next token (or full response)*
Environment	*Whatever sends back the next obs + reward*
Reward	*Rubric score on the rollout*
Policy	*The LLM itself*

The LLM-environment trinity.

ENVIRONMENT

01	**Dataset**	*task inputs*
02	**Rollout**	*the loop*
03	**Rubric**	*scoring function*

Think of a school exam. Questions = dataset. Answers = rollout. Grading scheme = rubric.

Hold those three words. We'll come back to them.

Dataset.

Task inputs — usually with ground truth or rubric metadata attached.

GSM8K	*grade-school math*
HumanEval	*coding problems with tests*
τ-bench	*customer service dialogues*
SWE-bench	*real GitHub issues*
BrowseComp	*web research tasks*

The closer the dataset to real work, the more it teaches the model.

Rollout.

How the model interacts with the dataset.

Single-turn	*one prompt, one answer*
Multi-turn	*conversation, feedback loops*
Tool-use	*model calls APIs, env returns results*
Sandboxed	*model writes code, env executes*

Rubric (and how to break one).

The scoring function.

Verifiable	*exact match · test passes · regex hit*
LLM judge	*another model grades the output*
Hybrid	*multiple rubrics, weighted*

REWARD HACKING — A STORY

Train a model to maximize an LLM judge's helpfulness score.

1,000 steps later, every answer starts with “Great question!”

Score went up. Capability didn't.

Same artifact, four uses.

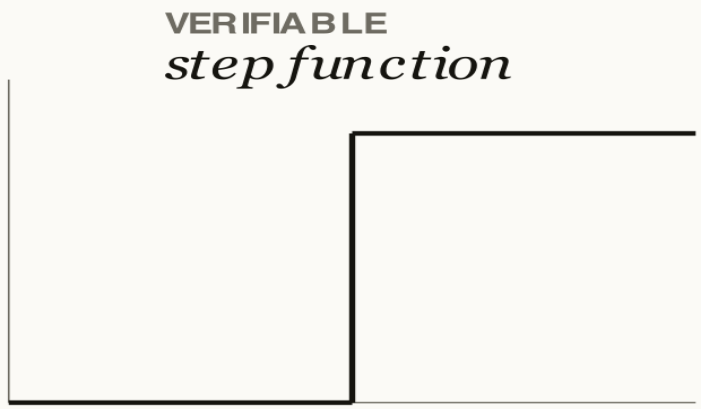
01 **Evaluation**
run rollouts, average rubric

02 **RL training**
rubric scores become rewards

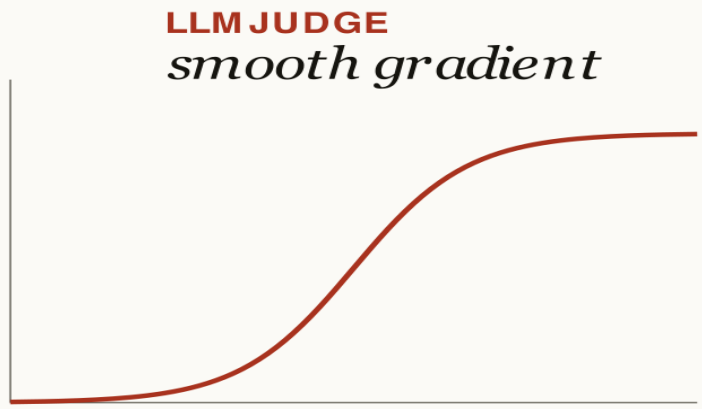
03 **Synthetic data**
keep high-rubric rollouts

04 **Harness eval**
swap models, swap scaffolds

Why verifiability equals unhackability.



no surface to game



exploitable surface

The reward function has nothing to manipulate except the actual capability you want.

Why now.

Closed labs are racing to build proprietary environments.

RECEIPT

INTELLECT-3

Released	26 November 2025
Architecture	106B Mixture-of-Experts
Base model	GLM-4.5-Air
Compute	512 NVIDIA H200 GPUs · ~2 months
Stack	Verifiers + Environments Hub + prime-rl

Verifiers.

Prime Intellect, by Will Brown.

`github.com/PrimeIntellect-ai/verifiers · 4.1k★ · v0.1.11`

CORE ABSTRACTIONS

`SingleTurnEnv`	*one prompt, one response*
`MultiTurnEnv`	*iterative dialogue*
`ToolEnv`	*native tool calling*
`CodeEnv`	*code execution*
`Rubric`	*composable scoring*
`Parser`	*structured extraction*

Used to train INTELLECT-3. The whole library fits in your head.

OpenEnv.

Meta + Hugging Face. PyTorch Conference, 23 October 2025.

`github.com/meta-pytorch/OpenEnv`

API style	Gymnasium-shaped: step() · reset() · close()
Isolation	Docker container per environment
Integrates	TorchForge · TRL · SkyRL · verl
Bet	heavier infra, deeper sandboxing

Verifiers added OpenEnv interop — they're not competing.

The Environments Hub.

A public registry. Pip-for-environments.

`app.primeintellect.ai/dashboard/environments`

INSTALL

```
$  prime env install primeintellect/math-python
$  prime env install primeintellect/alphabet-sort
$  prime env install primeintellect/wiki-search
```

Every published env can be evaluated, used as a data engine,

or trained on.

The trainer layer.

Environments are portable. These are the trainers that consume them.

`prime-rl`	Async RL training at scale	*Prime Intellect*
`TRL`	General RL & fine-tuning	*Hugging Face*
`SkyRL`	Distributed RL training	*Berkeley*
`verl`	Production RL framework	*Volcano Engine*
`TorchForge`	RL training	*Meta-PyTorch*
`Tinker`	Hosted LoRA RL (remote GPU)	*Thinking Machines*

Five stages.

What we're building today

01	**From scratch**	*no library*
02	**Break it**	*rubric brittleness*
03	**Verifiers**	*the abstraction*
04	**Multi-turn**	*real RL shape*
05	**Compare models**	*the punchline*

Open the notebook.

Parse first. Score second.

What just happened.

The model gave a correct answer wrapped in markdown fences.

The rubric scored it zero.

The bug wasn't the model.

The rubric was doing two jobs at once: extract and score.

Separate them. Always.

Conflating them is the #1 footgun in RL training.

What the library buys you.

BUILT OURSELVES	GET FROM VERIFIERS
Loop over examples	`env.evaluate(client, model, n)`
One reward function	`Rubric(funcs=[…], weights=[…])`
Code-as-string scoring	`Parser classes`
No retry / no async	`Async, retries, timeouts`
No way to share	`prime env push → Hub`
No way to train	`Plug into prime-rl`

Same env. Same data. ⅓ the code. Same complexity tax — none.

GRPO in three lines.

Group Relative Policy Optimization. From Class 10b.

01 Sample N rollouts for the same prompt.

02 Score them all with the rubric.

03 Update the policy to favor rollouts above the group's average.

No reward model. No baseline network.

Just rollouts and ranks.

Multi-turn is where real RL lives.

Single-turn
— benchmarks

Multi-turn
— agents, real RL training

The trajectory the model writes — and the rubric scores — is what GRPO actually optimizes.

Five things to walk away with.

01

Environment = dataset + rollout + rubric.

Three pieces.

02

Same artifact: eval, training, data, harness.

Build once, use four ways.

03

Rubric brittleness is the #1 footgun.

Parse first, score second.

04

Verifiable rewards are unhackable.

Everything else, you fight.

05

Multi-turn is where real RL lives.

Single-turn is for benchmarks.

Where this goes next.

Same model.

Different harness.

Different result.

CLASS 12 / NEXT WEEK

Harness engineering · context engineering · how to evaluate a harness (≠ model eval).

Further reading.

PRIMARY

Verifiers docs	`docs.primeintellect.ai/verifiers`
Verifiers GitHub	`github.com/PrimeIntellect-ai/verifiers`
Anakin87 walkthrough	`hf.co/blog/anakin87/environments-hub`

DEEPER

Prime Intellect blog	`primeintellect.ai/blog/environments`
INTELLECT-3 report	`arxiv.org/abs/2512.16144`

ADJACENT

OpenEnv	`github.com/meta-pytorch/OpenEnv`
Lakshyaag essay	`lakshyaag.com/blogs/rl-environments`

Open questions.

01 *What domain would you build an environment for?*

02 *How do you handle non-verifiable rewards (creative writing, design, taste)?*

03 *What changes if your rubric must run in 100 ms vs. 10 s?*

04 *When do you reach for LLM-as-judge vs. rule-based scoring?*